CORE JAVA SERIES — DAY 14

Generics, Wildcards & Type Erasure

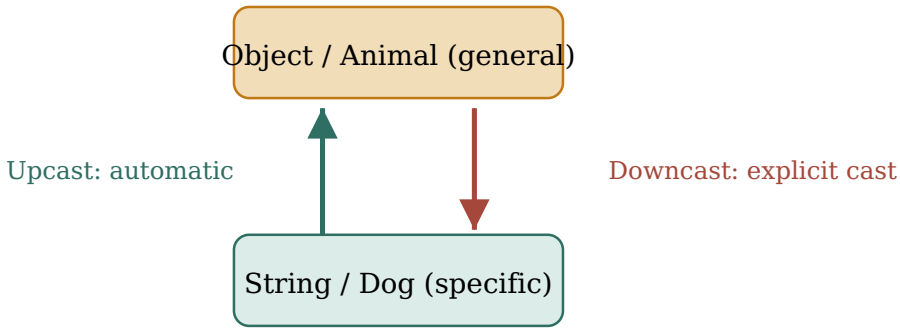
Why Java needs generics, how to write generic classes and methods, bounded types, why generics are invariant, wildcards (`? extends` / `? super`), the PECS rule, and what the JVM actually sees (type erasure).

Type Safety: The Foundation

Every value has a **type** — `10` is an int, `"Hello"` is a String, `true` is a boolean. A type is really a **set of rules** about what you may do with a value: ints support arithmetic; Strings support String-class methods; your own `Student` supports Student's methods. Java is a **typed language** precisely so these rules stop you from using values the wrong way (e.g. storing `10.2` in an int, or converting a String to an int directly).

Upcasting and Downcasting

With primitives, widening (int → long) is automatic and narrowing (long → int) needs an explicit cast. The same idea applies to class hierarchies:



Specific → general is always safe; general → specific is a gamble the compiler can't verify.

- **Upcasting** — specific type into a general (parent) type. No cast needed: `Animal a = new Dog();` and `Object obj = "Hello";`. Afterwards, through `a` you can call only **Animal's** methods (not `bark()`); through `obj` only Object's methods (`toString` , `equals` , `hashCode` …).
- **Downcasting** — general type back to a specific one. Needs an explicit cast: `String s = (String) obj;` Without it: "*cannot convert from Object to String.*"

> **Downcasting is risky.** With `Object obj = 10;` (auto-boxed to Integer), `String s = (String) obj;` compiles fine — the compiler can't know what `obj` really holds (it could come from user input, a file, an API) — but crashes at **run time** with `ClassCastException` . Casting between unrelated "sibling" classes (Integer ↔ String, both children of Object) is rejected outright at compile time.

Compile-Time Errors Beat Run-Time Errors

- Caught **early**, while writing code — bugs never reach production.
- The IDE flags them immediately (red underline), so there's far less debugging.
- Code is safer overall. As a developer, always push errors toward compile time.

Active Recall — Casting

Q1. Why is upcasting automatic but downcasting needs an explicit cast?

A. Every Dog is certainly an Animal, but an Animal/Object might not be a particular subtype — the compiler can't verify it, so you must declare intent.

Q2. When does ClassCastException occur?

A. At run time, when a downcast fails because the object's real class isn't the target type.

Q3. Why prefer compile-time errors?

A. They're caught early, flagged by the IDE, and need far less debugging than production failures.

The Problem Generics Solve

A `Box` class holding an `int` can't hold a String; making `BoxInt`, `BoxString`, `BoxDouble`… is endless duplication. The tempting fix: make the field an `Object`, since anything can be upcast to Object.

Why Object is a bad "universal" type:

- **Type information is lost** — the Box no longer knows if it holds an Integer, String or Boolean, so `b1.getValue() + 5` fails ("*operator + undefined for Object*").
- You must **downcast manually** every time (`(Integer) b1.getValue()`).
- One wrong cast → run-time `ClassCastException`, exactly the kind of error we want at compile time. Nothing stops you putting a String in a box you meant for ints.

Generic Classes

Replace the fixed type with a **type parameter** — a placeholder (conventionally `T`, but any name works) declared in angle brackets after the class name:

```
class Box<T> { private T value; Box(T value){ this.value = value; } T getValue(){ return value; } void setValue(T value){ this.value = value; } }
```

The real type is supplied when you create an object — the **type argument**:

```
Box<Integer> b1 = new Box<>(10);  Box<String> b2 = new Box<>("Hello");  Box<Boolean> b3 = new Box<>(false);
```

- **Type parameter** = what the class *expects* (the `T`); **type argument** = what you *pass* (`Integer`) — same vocabulary as method parameters/arguments.
- The type argument on the right-hand side (`new Box<>`, the "diamond") is optional — Java infers it from the left side. Leaving off type arguments entirely gives a "raw type" warning (Java treats it as Object).
- Generics only accept **reference types** — `Box<Integer>` works, `Box<int>` does not (reason in the type-erasure section).

What You Gain

- `b1.getValue() + 5` just works (15) — **type information is preserved**, no manual cast.
- `String s = b1.getValue();` is a **compile-time** error; `(String) b1.getValue()` is also rejected (Integer and String are unrelated siblings). So `ClassCastException` from misusing a generic type can't occur.
- Only operations valid for the actual type are allowed (e.g. `b2.getValue().substring(1)` works for a String box but not for an Integer box).

Multiple Type Parameters

`class Pair<T, U> { T first; U second; Pair(T first, U second){...} }` → `Pair<Integer, String> p = new Pair<>(23, "Aditya");` — the two types need no relationship to each other.

Generic Methods

Declare the type parameters **before the return type**:

```
static <T> T getResult(T x) { return x; }  static <T, U> void printPair(T first, U second) { ... }
```

Type inference: when calling, you normally don't state the types — Java infers them from the arguments (`printPair(11, "Hello")` → T=Integer, U=String). A generic method gives compile-time safety where the Object version would risk `ClassCastException` (`String y = getResult(23)` is a compile error).

Bounded Types (Upper Bounds on T)

Inside `Box<T>`, Java only knows `T` is "some Object", so you can only call Object's methods on it (`value.doubleValue()` fails). Restrict T with `extends`:

```
class Box<T extends Number> { T value; void printDouble(){ System.out.println(value.doubleValue()); } }
```

- T must now be `Number` or a subclass (`Integer`, `Double`, `Float`, `Long` ...) — so Number's methods like `doubleValue()` are callable.
- `Box<String>` is now a compile error ("*bound mismatch*").
- Here `extends` covers both inheritance and interface implementation.

Multiple Bounds

`class Box<T extends Animal & Swimmable>` — T must extend `Animal` **and** implement `Swimmable`. Only `Fish` qualifies (Dog and plain Animal fail). **Rule:** the class (if any) goes first, then interfaces; you can list several interfaces joined by `&`.

Active Recall — Generics Basics

Q1. Why is a Box with an Object field a poor general-purpose container?

A. It loses type information, forces manual downcasts, and lets wrong types slip through to a run-time ClassCastException.

Q2. Difference between a type parameter and a type argument?

A. The parameter is the placeholder in the declaration (T); the argument is the concrete type supplied at use (Integer).

Q3. Where does a generic method's type-parameter list go?

A. Right before the return type: `static <T> T getResult(T x)`.

Q4. What does `T extends Number` achieve?

A. Restricts T to Number and its subtypes, so Number's methods (e.g. doubleValue) can be called on T, and unrelated types like String are rejected at compile time.

Q5. In `T extends Animal & Swimmable`, which comes first — class or interface?

A. The class (Animal) first, then the interfaces.

Generics Are Invariant

Since `Dog` extends `Animal`, `Animal a = new Dog();` works. But generics **break** the parent-child relationship:

`List<Dog> dogs = new ArrayList<>(); List<Animal> animals = dogs;` → **compile error** (*cannot convert from List<Dog> to List<Animal>*). "Dog is an Animal" does **not** mean "List of Dog is a List of Animal."

Formally: **Generics are invariant** — even if A is a subtype of B, `Generic<A>` is *not* a subtype of `Generic<B>`.

Why? Protecting Type Safety

If it were allowed, you could write `animals.add(new Animal())` through the `List<Animal>` reference — silently putting a plain Animal (or a Cat) into what is really a list of Dogs. Later, code that loops over the dog list calling `dog.bark()` would blow up. Reading would be fine (every element is at least an Animal); **writing** is what breaks safety.

Contrast: Arrays Are Covariant (and Unsafe)

`Dog[] dogs = new Dog[10]; Animal[] animals = dogs;` compiles, because arrays are **covariant**. Then `animals[4] = new Animal();` also compiles — but throws `ArrayStoreException` at **run time** (the slot is really a Dog reference). Generics choose the safe route: reject it at **compile time**.

Invariance has a practical cost: a method `void printAll(List<Animal> list)` cannot be called with a `List<Dog>`, even though every dog is an animal. Wildcards fix this.

Wildcards: the `?`

A wildcard `?` means "some unknown type." `List<?>` can reference a list of *anything* — Dogs, Animals, Strings, Integers:

> `static void fun(List<?> values)` accepts `List<Dog>`, `List<Animal>`, `List<Integer>`...

- **Writing:** not allowed (except `null`). `values.add("Hello")`, `values.add(10)`, even `values.add(new Dog())` are all compile errors — the compiler doesn't know what the list really holds, and the call could come from anywhere.
- **Reading:** only into `Object` (`Object o = values.get(0);`) — the only type that's safe for every possible list. You can still call Object methods such as `o.getClass().getName()`.

Plain `?` is very flexible but very limited: you can't add anything and can only read as Object.

Upper-Bounded Wildcard: `? extends Animal` (Reading)

`List<? extends Animal>` accepts a list of **Animal or any subtype** (Dog, Cat, Labrador…) — but not Strings or Integers.

- **Reading is safe:** whatever the list really contains, every element is at least an Animal, so `Animal a = values.get(0);` and a `for (Animal a : values) a.eat();` loop work with no cast and no risk of ClassCastException.
- **Writing is still blocked:** if the caller passed a `List<Cat>`, adding a Dog would be nonsense (cat and dog are siblings); if they passed `List<Dog>`, adding a plain Animal would put a parent into a child's list. Compiler can't tell which, so it forbids all adds.

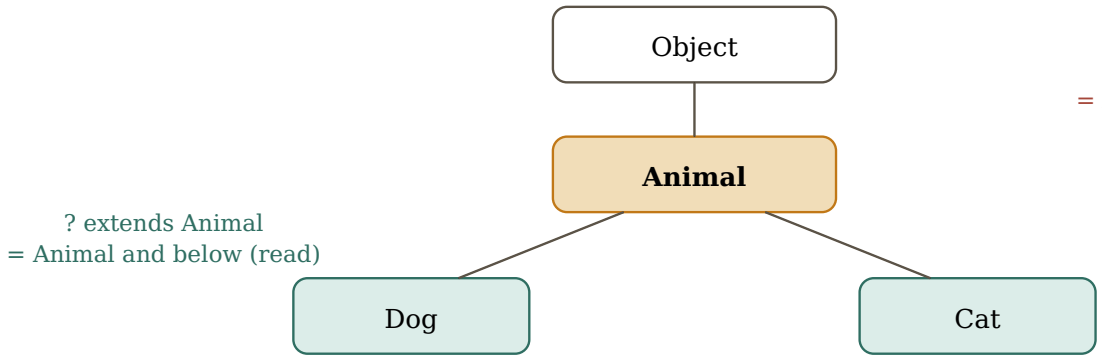
This behavior is **covariance** (reading).

Lower-Bounded Wildcard: `? super Animal` (Writing)

`List<? super Animal>` accepts a list of **Animal or any supertype** (Animal, Object) — not Dog or Cat.

- **Writing is safe:** the list is at least a list of Animals, so you can always add an Animal — and any subtype (Dog, Cat, Labrador, GermanShepherd), since each is an Animal and an Object. Whether the caller passed `List<Animal>` or `List<Object>`, it works.
- **Reading loses type info:** an element could be an Animal or merely an Object, so you can only read as `Object`; `Animal a = values.get(0)` won't compile.

This behavior is **contravariance** (writing).



extends = Animal downwards (safe to read as Animal); super = Animal upwards (safe to write Animals in).

The PECS Rule

> **P**roducer **E**xtends, **C**onsumer **S**uper. If a structure **produces** data you read from, use `? extends`. If it **consumes** data you write into it, use `? super`.

Form	Accepts	Read	Write	Variance
List<T> (invariant)	exactly T	as T	T	Invariant
List<?>	any list	Object only	nothing (null only)	—
List<? extends Animal>	Animal & subtypes	as Animal	nothing	Covariant
List<? super Animal>	Animal & supertypes	Object only	Animal & subtypes	Contravariant

T vs ? — When to Use Which

- `T` = a type parameter: declared on a generic class or method and used to *define* it. Class definition and instantiation use real types, never `?` (`Box<?> b = new Box<?>()` is a syntax error), and a method's declared type parameter must be `T`, not `?`.
- `?` = a wildcard: used where a generic *container is referred to* — typically method parameters or reference variables — to relax the invariance rule.

Active Recall — Wildcards

Q1. Why can't a List<Dog> be assigned to a List<Animal>?

A. Generics are invariant; allowing it would let you add a non-Dog into a list of Dogs through the Animal reference, breaking type safety.

Q2. What happens with arrays in the same situation?

A. Arrays are covariant, so it compiles, but storing the wrong type throws ArrayStoreException at run time.

Q3. With List<? extends Animal>, what can you read and write?

A. Read elements as Animal safely; write nothing (covariant, "producer").

Q4. With List<? super Animal>, what can you read and write?

A. Write Animal and any subtype (Dog, Cat…); read only as Object (contravariant, "consumer").

Q5. What does PECS stand for?

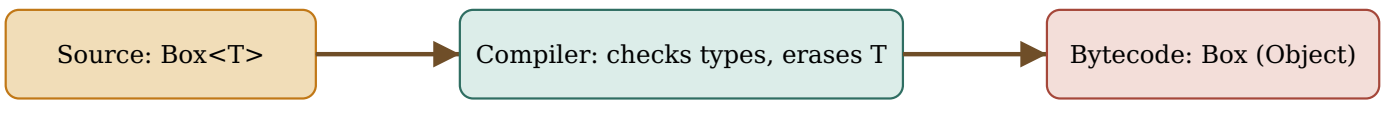
A. Producer Extends, Consumer Super.

Q6. Can you write `new Box<?>()`?

A. No — wildcards are for referring to generic containers (e.g. parameters), not for instantiating or declaring type parameters.

Type Erasure: What the JVM Actually Sees

Does the JVM know about generics? **No.** Generics are purely a **compile-time safety layer**. After the compiler checks types, it **erases** the generic information and produces ordinary bytecode; the JVM never sees `<T>`.



By run time, the generic information is gone.

Erasure Rules

1. **Unbounded T** → replaced with `Object`. (`class Box<T>` becomes a Box with an Object field.)
2. **Bounded T** → replaced with the bound. (`T extends Number` becomes `Number`; with multiple bounds, the first.)
3. **Casts are inserted automatically:** your `Integer x = b1.getValue();` becomes `Integer x = (Integer) b1.getValue();` in bytecode. Because the compiler already verified that only Integers could have gone into `Box<Integer>`, this cast can never fail — it's just a formality Java requires when going from Object to a specific type.

Consequences of Erasure

- **No `instanceof` with generic types:** `list instanceof List<String>` is illegal — at run time it's just a `List`.
- **No overloading by generic argument:** `print(List<String>)` and `print(List<Integer>)` both erase to `print(List)` — identical signatures, so it won't compile.
- **Compiler-generated bridge methods:** if `class Child extends Parent<String>` overrides `String get()`, erasure turns the parent's method into `Object get()`, so the override no longer matches. The compiler quietly generates a synthetic **bridge method** `Object get()` in Child that calls `String get()`, preserving polymorphism and inheritance.
- **No primitives as type arguments:** erasure replaces T with Object (or a bound), but `int` is not an Object — it lives on the stack and isn't a class. Wrapper classes (`Integer` etc.) are, so use those.

Why Does Erasure Exist?

Generics arrived in **Java 5**. Before that, everyone wrote `List l = new ArrayList();`. Making the JVM generics-aware would have changed how it reads class files and broken the enormous body of existing bytecode. Instead, Java made generics a **compiler-only safety check** that rewrites everything into the old form, so old and new bytecode run identically on the JVM — full backward compatibility.

Active Recall — Type Erasure

Q1. Does the JVM know about generics at run time?

A. No — generics are erased by the compiler; the JVM sees only ordinary (raw) types.

Q2. How does erasure treat T, and T extends Number?

A. Unbounded T → Object; bounded T → its bound (Number).

Q3. Why can the compiler-inserted cast never throw ClassCastException?

A. The compiler already guaranteed, at compile time, that only the declared type argument could have been stored.

Q4. Why can't you overload print(List<String>) and print(List<Integer>)?

A. After erasure both become print(List) — identical signatures.

Q5. What is a bridge method?

A. A compiler-generated method that bridges an erased parent signature (e.g. Object get()) to the child's overriding method (String get()), keeping polymorphism working.

Q6. Why aren't primitives allowed as generic type arguments?

A. Erasure replaces T with Object, which a primitive like int cannot be; use wrappers like Integer.

Q7. Why did Java choose erasure at all?

A. Backward compatibility: generics arrived in Java 5, and changing the JVM would have broken all existing bytecode.